The Nanz Language Book — v4

MinZ Project

2026-03-13

Contents

The Nanz Language Book — v4

Modern language. Vintage iron. Zero overhead.

Nanz is a statically typed systems language that compiles to Z80 assembly with no runtime, no garbage collector, and no performance tax. Every abstraction — iterators, lambdas, interfaces, closures — disappears at compile time and leaves only tight machine code.

Also targets native AMD64 via C99 and QBE, and MOS 6502.

Version: MinZ compiler v0.19.6+ (commit 973bf50) **Date:** 2026-03-13 **Status:** Core features stable · 23/23 showcase · 20/20 test packages

---

Table of Contents

Chapter 1: What is Nanz?

Nanz (`.nanz`) is the active frontend language of the **MinZ compiler system**. It targets the MIR2 backend — a modern, SSA-like intermediate representation with:

- Interprocedural calling convention optimization (PFCCO)
- PBQP register allocator (cost-weighted physical register assignment)
- Pre-allocation coalescing (block-parameter register unification)
- LUT synthesis (pure functions with bounded inputs → lookup tables)
- Compile-time assertion evaluation on both the MIR2 VM *and* the Z80 binary
- A Z80 emulator used as a constant evaluator *inside* the compiler
- Multiple backends: Z80 (production), MOS 6502, C99, QBE (AMD64/ARM64/RISC-V)

1.1 The Compilation Pipeline

```
source.nanz
    │
    ▼  nanz.Parse()
*hir.Module         ← High-level IR: structured control flow, named vars
    │
    ▼  hir.LowerModule()
*mir2.Module (raw)      ← SSA-like virtual registers, typed ops
    │
    ▼  Optimization passes
*mir2.Module (opt)    ← Constants folded, dead stores removed, LUTs generated,
    │                     branches eliminated, conditional returns sunk
    ▼  Compile-time assertions (MIR2 VM)
    │               ← Each assert fn(args)==expected runs on the MIR2 VM
    ▼  Contract optimization (PFCCO)
    │                   ← Interprocedural calling convention selection
    ▼  PreallocCoalesce
    │                   ← Block-param → block-arg register unification
    ▼  PBQP register allocation
    │                ← Virtual regs → physical Z80 regs (cost-weighted)
    ▼  Post-allocation coalescing
    │               ← Eliminate redundant moves across block boundaries
    ▼  Z80Codegen
source.a80              ← MZA-compatible Z80 assembly text
    │
    ▼  Compile-time assertions (Z80 binary)
    │               ← Same asserts now run on the real assembled binary
    ▼  mza (MZA assembler)
source.bin / .tap       ← Ready to run on Z80 hardware or emulator
```

The compiler runs assertions **twice**: once on the abstract MIR2 VM (fast, catches algorithm bugs) and once on the assembled Z80 binary (catches codegen bugs). If both pass, the function is correct by construction.

1.2 Nanz vs. MinZ

MinZ (`.minz`) is the original frontend, targeting MIR1 + an older codegen. That pipeline is **frozen** — it works but is not developed further.

Nanz is the **replacement**: same syntax spirit, radically better backend. New programs go in Nanz.

Features only in Nanz: PFCCO contracts, PreallocCoalesce, dual-VM asserts, as cast, `mzn` native backend, signed comparison, trivial inliner, ForEachEdge, LUTGen, BranchEquiv, CondRetSink+CmpSubCarry, `@smc` parameters, 6502 backend.

Features only in MinZ (frozen): Enums, string interpolation, `@error` propagation, `@define` macros, `@if/@elif` conditional compilation, `import` system. See Feature Gap Analysis for the parity roadmap.

1.3 Nanz vs. PL/M-80

PL/M-80 (`.plm`) is an Intel language from the 1970s, used to write CP/M and early microcomputer software. The MinZ compiler includes a complete PL/M-80 parser that compiles PL/M programs through the **same HIR→MIR2→Z80 pipeline** as Nanz.

This means: - 26/26 Intel PL/M-80 Tools reference files parse successfully (100%) - 1338 functions, 943 globals, 11661 statements → HIR → Z80 - PL/M programs benefit from PBQP allocation, LUTGen, and all MIR2 passes

1.4 Design Philosophy

No runtime system. No garbage collector. No dynamic dispatch vtables. Every abstraction is transparent at compile time: lambdas become inline code, interfaces become direct function calls, iterators become DJNZ loops.

Provable by construction. Compile-time assertions checked on two independent VMs catch two classes of bugs that historically slip through: - *Algorithm bugs*: both VMs produce the wrong answer - *Codegen bugs*: the MIR2 VM gives the right answer; the Z80 binary diverges

Target-honest. The optimizer knows it is targeting Z80. It knows that `SUB B` followed by `RET NC` is shorter than a branch. It uses the Z80 carry flag to communicate comparison results. It emits `DJNZ` instead of `DEC B / JR NZ`. These are not peepholes applied after the fact — they emerge from the MIR2 pass structure.

Multi-target. The same Nanz source can compile to Z80 assembly (production), MOS 6502 assembly (retro), C99 (verification), and QBE IL (native AMD64/ARM64). All backends consume the same MIR2 IR.

Chapter 2: Syntax Reference

2.1 Module Structure

A Nanz source file is a **module**: a flat sequence of top-level declarations that may appear in any order. No imports, no forward declarations required.

```
// Declarations may appear in any order.
struct Vec2 { x: u8, y: u8 }

global origin: Vec2

fun Vec2.add(self: ^Vec2, other: Vec2) -> Vec2 { ... }
```

```
interface Shape { area }

fun area(s: Shape) -> u16 { ... }

assert area_of_unit_square() == 1
```

Comments: `//` line, `/* */` block.

2.2 Functions

```
fun name(param1: Type1, param2: Type2) -> ReturnType {
    // body
}
```

`fn` is accepted as an alias for `fun`. Functions with no return value use `void` or omit the `->` clause entirely:

```
fun clear(buf: ^u8, n: u8) {
    var i: u8 = 0
    while i < n {
        buf[i] = 0
        i = i + 1
    }
}
```

Multiple return values:

```
fun swap(a: u8, b: u8) -> (u8, u8) {
    return (b, a)
}

fun divmod(a: u8, b: u8) -> (u8, u8) {
    return (a / b, a % b)
}

// Call site:
let (q, r) = divmod(10, 3)    // q=3, r=1
let (_, r2) = divmod(10, 3)   // discard quotient with _
```

Return values are assigned to registers: pos0→HL (or A for u8), pos1→DE, pos2→B. The `_` blank identifier triggers dead store elimination — the discarded value is never computed.

Operator overloading — operators are functions with the operator symbol as name:

```
fun +(a: Vec2, b: Vec2) -> Vec2 {
    return Vec2{ x: a.x + b.x, y: a.y + b.y }
}
// Now: a + b → op_add(a, b) → Vec2_add-style call
```

Struct methods — namespaced with `Type.method`:

```
fun Vec2.scale(self: ^Vec2, factor: u8) -> Vec2 {
    return Vec2{ x: self.x * factor, y: self.y * factor }
}
// v.scale(3) → Vec2_scale(&v, 3)  — UFCS, zero cost
```

2.3 Variables

```
var i: u8 = 0       // explicit type, optional initializer
let x = 42          // type inferred (u8)
let y: u16 = 1000   // explicit type overrides inference
```

Use-before-init warning: The compiler tracks uninitialized variables at parse time and emits warnings on use:

```
var ptr: ^u8          // no initializer
let v = ptr^          // ⚠ warning: ptr used before initialization
```

This catches a whole class of bugs that would be silent in C.

2.4 Global Variables

```
global counter: u8 = 0
global screen: [u8; 6912] at(0x4000)   // hardware-mapped ZX Spectrum VRAM
global palette: Color at(0xFF00)        // peripheral mapped at fixed address
```

The `at(addr)` clause maps the global to a specific Z80 address — no pointer arithmetic needed.

2.5 Control Flow

```
// if / else
if x > 0 {
    do_positive(x)
} else {
    do_non_positive()
}

// while
while i < n {
    process(arr[i])
    i = i + 1
}

// for i in range
for i in 0..n {
    process(i)
}

// for each element in array
for x in buf[0..n] {
    process(x)
}

// switch
switch state {
    case 0: idle()
    case 1: run()
    default: error()
}
```

```
// break / continue
while true {
    if done { break }
    if skip { continue }
    work()
}
```

2.6 Pointers and Arrays

```
var p: ^u8              // typed pointer to u8
var q: ptr              // untyped pointer

let val = p^            // dereference → u8
let elem = p[3]         // index (equivalent to (p+3)^) → u8
p[0] = 42               // store through pointer

let addr = &my_global   // address-of
```

Pointer arithmetic is done at the Z80 level via `LD HL` + `ADD HL,DE`. The programmer does not write offset calculations manually — the compiler emits them from `ptr[i]`.

2.7 Structs

```
struct Color {
    r: u8
    g: u8
    b: u8
}

struct Vec3d {
    x: u16
    y: u16
    z: u8       // z.Offset = 4 (computed at parse time from field layout)
}

global sky: Color

fun set_sky(r: u8, g: u8, b: u8) {
    sky = Color{ r: r, g: g, b: b }
}
```

Field offsets are computed at parse time from the struct declaration. Mixed-width structs (u8 + u16) lay out correctly with byte-accurate offsets.

Consecutive field stores are fused into an HL-chain: `LD HL, &sky / LD (HL), r / INC HL / LD (HL), g / INC HL / LD (HL), b` — 53T vs. 61T for three separate absolute stores. This optimization fires automatically when fields are stored in declaration order.

2.8 Lambdas

```
// Inline lambda expression
let double = |x: u8| { return x * 2 }

// Lambda used in iterator chain (fused — no CALL emitted)
arr.map(|x: u8| x * 2).forEach(|x: u8| process(x), n)

// Lambda capturing outer variable (zero-cost — threaded as block param)
var sum: u8 = 0
arr.forEach(|x: u8| { sum = sum + x }, n)
// sum is threaded through the DJNZ loop as a register — no heap, no spill
```

2.9 Casts

Nanz supports two equivalent cast syntaxes:

```
// Function-style cast (original syntax)
let byte = u8(some_u16)     // truncate: take low byte
let word = u16(some_u8)     // zero-extend to 16 bits
let signed = i8(some_u8)    // reinterpret (same bits, signed semantics)

// "as" cast (added in v4)
let byte = some_u16 as u8   // same as u8(some_u16)
let word = some_u8 as u16   // same as u16(some_u8)
let signed = some_u8 as i8  // same as i8(some_u8)
```

Both forms produce the same HIR `CastExpr` and generate identical code. Use whichever reads better in context — `as` is cleaner in chains: `(a + b) as u16 * 256`.

2.10 Extern Functions

```
@extern fun rom_print(s: ptr) -> void           // resolved at link time
@extern(0x0010) fun rst_10h(a: u8) -> void      // RST 0x10 (single byte CALL)
@extern(0xBB00) fun bc_sendchar(c: u8) -> void  // CALL 0xBB00 (CP/M BDOS-style)
```

`@extern(addr)` functions with `addr` that is a multiple of 8 and ≤ 0x38 emit `RST n` (1 byte, 11T). All other `@extern(addr)` functions emit `CALL addr` (3 bytes, 17T). The compiler selects the cheaper form automatically.

2.11 Register Annotations on Parameters

```
fun fast_op(@z80_a x: u8, @z80_b count: u8, @z80_hl ptr: ^u8) -> u8 { ... }
```

Available annotations: `@z80_a`, `@z80_b`, `@z80_c`, `@z80_hl`, `@z80_de`. These override the PBQP allocator's choice for that parameter — use only when calling from hand-written assembly that has specific register constraints.

2.12 Ranged Types

```
fun double_angle(a: u8<0..359>) -> u16<0..718> { return u16(a) * 2 }
```

`u8<lo..hi>` declares a parameter or return with a guaranteed value range. The compiler uses this to: 1. Verify the range at call sites (static check, no runtime

overhead) 2. Auto-generate a **lookup table** when the range is small enough (≤ 256 values, pure function)

LUT generation example:

```
fun sin_table(angle: u8<0..255>) -> u8 { ... }
// → Evaluates sin_table(0..255) at compile time via MIR2 VM
// → Emits: sin_table_lut: DB 0, 1, 3, 6, 9, ... (256 bytes)
// → Function body replaced by table lookup: LD HL, sin_table_lut / LD D,0 / LD E,angle
// → Runtime cost: 6 instructions, ~39T — no computation at all
```

2.13 Interfaces and UFCS

```
interface Animal {
    speak
    eat
}

fun Dog.speak(self: Dog) { ... }
fun Cat.speak(self: Cat) { ... }

// Interface as parameter type — monomorphized at compile time:
fun feed(a: Animal) {
    a.speak()   // → Dog_speak(a) or Cat_speak(a) depending on concrete type
}
```

Cost: zero. No vtable. No fat pointer. The concrete type is resolved at compile time. `a.speak()` emits a direct `CALL Dog_speak` or `CALL Cat_speak`.

UFCS — uniform function call syntax — lets you write `a.method(args)` for any function `fun Type.method(self: Type, args)`. It desugars at parse time:

```
v.scale(3)   →   Vec2_scale(&v, 3)   →   CALL Vec2_scale
```

2.14 Inline Assembly

```
fun fast_clear() {
    asm {
        XOR A
        LD (HL), A
        INC HL
        DJNZ -3
    }
}

// Target-gated: only included for Z80 backend
fun platform_init() {
    asm(z80) {
        DI
        LD SP, 0xFFFF
        EI
    }
}
```

Inline assembly blocks emit Z80 instructions verbatim. The `asm(z80)` variant is only included when compiling for Z80 — `asm(mir2)` runs on the MIR2 VM instead.

Chapter 3: Type System

3.1 Numeric Types

Type	Width	Description
u8	8-bit	Unsigned byte — Z80 registers A/B/C/D/E/H/L
u16	16-bit	Unsigned word — Z80 register pairs HL/DE/BC
i8	8-bit	Signed byte (same registers, signed arithmetic)
i16	16-bit	Signed word
u24	24-bit	24-bit unsigned (eZ80 / Agon Light 2 native)
u32	32-bit	32-bit via Z80 EXX shadow pair (HL'/DE'/BC')
i32	32-bit	Signed 32-bit via shadow pair
f8.8	16-bit	Fixed-point: 8 integer bits + 8 fractional bits
f16.8	24-bit	Fixed-point: 16 integer + 8 fractional
f8.16	24-bit	Fixed-point: 8 integer + 16 fractional
f16.16	32-bit	Fixed-point: 16 integer + 16 fractional
bool	8-bit	false=0, true≠0
void	—	Return type only

Fixed-point types parse with dot notation: `f8.8` is "f" followed by "8" (integer bits) "." "8" (fractional bits). The compiler handles arithmetic (add/sub exact, mul/div require shifts that the optimizer emits).

3.2 Signed Arithmetic

Signed types (`i8`, `i16`) use the same physical registers as unsigned but with signed comparison semantics:

```
fun max_i8(a: i8, b: i8) -> i8 {
    if a > b { return a }
    return b
}

assert max_i8(5, 3) == 5         // both positive
assert max_i8(251, 5) == 5       // 251 = -5 in i8, so max(-5, 5) = 5
assert max_i8(251, 253) == 253   // max(-5, -3) = -3 = 253
```

How it works: Values are stored truncated — `i8(-5)` is 251 in memory. The lowerer picks `CmpGe` for signed and `CmpUge` for unsigned (via `IsSigned()`). The MIR2 VM sign-extends both operands before signed comparison: `signExtend(251, 8) = -5`. This makes `(-5) < 5` evaluate to `true`.

On Z80 hardware, signed comparison uses the S^V flag combination (sign XOR overflow) — the same approach the Z80 was designed for.

3.3 Pointer Types

Type	Description
`ptr`	Untyped 16-bit Z80 address
`^T`	Typed pointer to T (human-readable; same as `ptr` at machine level)

`^Struct` pointer receivers enable clean method syntax:

```
fun Acc.add(self: ^Acc, amount: u8) -> u8 {
    self.val = self.val + amount
    return self.val
}
// self^.val also works (explicit dereference)
```

3.4 The u32 "ClassDWord" via EXX Shadow Pair

32-bit values on Z80 — without a 32-bit bus — use the **EXX shadow register pair** trick:

```
fun add32(a: u32, b: u32) -> u32 { return a + b }
```

Generated Z80:

```
; fun add32(a: u32 = HL/DE', b: u32 = HL'/DE) -> u32 = HL/DE
add32:
    ADD HL, DE      ; low 16 bits: HL += DE
    EXX             ; swap to shadow pair
    ADC HL, DE      ; high 16 bits: HL' += DE' + carry
    EXX
    RET
```

5 instructions. The PBQP allocator places the 32-bit halves in the main and shadow HL — they don't interfere with each other because EXX separates them.

3.5 Calling Convention

The calling convention is **not fixed** — it is computed per function by the PBQP register allocator and interprocedural contract optimizer (PFCCO). Typical Z80 mapping:

Class	Z80 register	Typical use
ClassAcc	A	First u8 param, return value
ClassCounter	B	Second u8 param, loop counter
ClassPointer	HL	u16 params, pointer args, return value
ClassIndex	DE	Second u16 param
ClassPair	BC	Third param or general pair
ClassGeneral	C/D/E/H/L	Remaining 8-bit params
ClassDWord	HL+shadow	u32 values

The **contract optimizer** (PFCCO) searches across the call graph for the assignment that minimizes total T-states for caller+callee. The result is that calling conventions are tailored to the specific set of functions in your program — not a fixed ABI.

Example: If function `f(a, b)` always calls `g(b)`, PFCCO will assign `b` to the same register in both functions, eliminating the move at the call site. This is computed globally, not locally — the optimizer sees the entire call graph.

Chapter 4: Structs, Methods, and Interfaces

4.1 Struct Declaration and Layout

```
struct Sprite {
    x: u8       // offset 0
    y: u8       // offset 1
    frame: u8   // offset 2
    tile: u8    // offset 3
}
```

The compiler computes byte offsets at parse time: `x` at 0, `y` at 1, `frame` at 2, `tile` at 3. Mixed-width structs with u16 fields get 2-byte offsets:

```
struct Vec3d {
    x: u16   // offset 0
    y: u16   // offset 2
    z: u8    // offset 4
}
```

4.2 Methods and UFCS

```
fun Sprite.move(self: ^Sprite, dx: i8, dy: i8) {
    self.x = u8(i8(self.x) + dx)
    self.y = u8(i8(self.y) + dy)
}

// Call site:
sprite.move(+1, 0)   →   Sprite_move(&sprite, 1, 0)   →   CALL Sprite_move
```

Zero overhead. The method table exists only in the parser — no runtime representation.

4.3 Interfaces and Static Dispatch

```
interface Drawable {
    draw
}

fun Circle.draw(self: ^Circle) { ... }
fun Rect.draw(self: ^Rect) { ... }

fun render_all(shape: Drawable) {
    shape.draw()
}

// render_all(my_circle):
//   → only one implementation of 'draw' for Circle exists
//   → monomorphized to: CALL Circle_draw
```

When multiple implementors exist, the compiler requires a statically known concrete type at the call site. When a function parameter is typed as an interface and only one implementor exists in the module, the call is **monomorphized automatically** — no code change required.

4.4 Operator Overloading

```
struct Vec2 { x: u8, y: u8 }

fun +(a: Vec2, b: Vec2) -> Vec2 {
    return Vec2{ x: a.x + b.x, y: a.y + b.y }
}

let v1 = Vec2{ x: 10, y: 20 }
let v2 = Vec2{ x: 5, y: 3 }
let v3 = v1 + v2   // → op_add(v1, v2) → Vec2_add-like dispatch
```

Operators for primitive types (`u8 + u8`) use Z80 ALU instructions directly — overloading only fires when one operand is a struct type.

Vec3 with operator overloading — a 3D wireframe building block:

```
struct Vec3 { x: i8, y: i8, z: i8 }

fun +(a: Vec3, b: Vec3) -> Vec3 {
    return Vec3 { x: a.x + b.x, y: a.y + b.y, z: a.z + b.z }
}

fun midpoint(a: Vec3, b: Vec3) -> Vec3 {
    return Vec3 {
        x: (a.x + b.x) >> 1,   // division by 2 via arithmetic shift
        y: (a.y + b.y) >> 1,
        z: (a.z + b.z) >> 1
    }
}
```

Zero-cost: the compiler inlines structs into registers. PFCCO picks the optimal layout (`x→H, y→L, z→D` or whatever minimizes total moves).

Chapter 5: Iterator Chains

Nanz supports a composable iterator chain syntax on arrays and pointers. The crucial property: **the chain is fused at compile time into a single DJNZ loop**. No intermediate arrays. No function pointer overhead. No virtual dispatch.

5.1 Combinators

Method	Meaning
`ptr.map(λ)`	Transform each element
`ptr.filter(λ)`	Keep elements where λ is true

Method	Meaning
`ptr.forEach(λ, n)`	Execute λ for each of n elements
`ptr.fold(init, λ)`	Reduce n elements to a single value
`ptr.reduce(λ)`	Reduce with first element as init
`ptr.take(k)`	Keep first k elements
`ptr.skip(k)`	Skip first k elements
`ptr.enumerate()`	Add element index
`ptr.chain(other)`	Concatenate two iterators

5.2 Fusion

```
arr.map(|x: u8| x * 2).filter(|x: u8| x > 10).forEach(|x: u8| process(x), n)
```

The parser recognizes this chain pattern. The HIR lowerer's `recognizeIterChain` function fuses all stages before emitting any MIR2 instructions. The result is a single loop:

```
.loop:
    LD A, (HL)          ; load element
    INC HL              ; advance pointer
    ADD A, A            ; map: x * 2  (strength-reduced from MUL)
    CP 11               ; filter: x > 10 → x >= 11
    JR C, .skip         ; skip if filter fails
    CALL process        ; forEach body
.skip:
    DJNZ .loop          ; B--; branch if B ≠ 0
```

No intermediate storage. No lambda calls. The filter check and map transform are inlined.

5.3 Closure Capture

Lambdas inside chains can capture and mutate outer variables:

```
var sum: u8 = 0
arr.forEach(|x: u8| { sum = sum + x }, n)
```

`sum` is not spilled to memory. It is threaded through the DJNZ loop as a **block parameter** — a loop-carried SSA value that lives in a register:

```
    LD B, n             ; counter
    LD C, 0             ; sum = 0 (in C)
.loop:
    LD A, (HL)          ; x
    ADD A, C            ; sum + x
    LD C, A             ; sum = result
    INC HL
    DJNZ .loop
    LD A, C             ; move sum to A for return/use
```

This is **zero-cost closure capture** — `sum` is a CPU register throughout the loop.

5.4 E2E Verification

All 11 iterator chain combinations are verified end-to-end:

Chain	Binary	T-states
forEach	hex-verified	~43T/elem
map+forEach	hex-verified	~50T/elem
filter+forEach	hex-verified	~52T/elem
map+filter+forEach	hex-verified	~57T/elem
take+forEach	hex-verified	~43T/elem
skip+forEach	hex-verified	~43T/elem
lambda map	hex-verified	~50T/elem
lambda filter	hex-verified	~52T/elem
multi-stage	hex-verified	varies
fold	hex-verified	~30T/elem
forEach with capture	hex-verified	~30T/elem

"Hex-verified" means: compiled to Z80 binary, loaded into the MZE emulator, executed, result checked against expected value.

Chapter 6: `range(lo..hi)` — Counter-Based Iteration

6.1 Overview

`range(lo..hi)` is a **counter-based iterator source** — no memory pointer, no `LD A,(HL)`, no `INC HL`. Elements are the DJNZ counter value itself, counting down from `hi−lo` to 1.

```
fun sum_range(n: u8) -> u8 {
    return range(0..n).fold(0, |acc: u8, i: u8| { return acc + i })
}
```

Generated Z80:

```
; fun sum_range(n: u8 = A) -> u8 = A
sum_range:
    LD B, 0              ; acc init = 0
    AND A                ; pre-check: n == 0?
    JRS NZ, .trmp0
    JRS .rng_exit2
.rng_body1:
    ADD A, B             ; acc += counter  ← ONE instruction per iteration
    DJNZ .rng_body1      ; B--; branch if B ≠ 0
    LD B, A
    JP .rng_exit2
.rng_exit2:
    LD A, B
    RET
.trmp0:
    LD C, A              ; save n (parallel copy resolver)
    LD A, B              ; A = 0 (acc init)
    LD B, C              ; B = n (counter)
    JRS .rng_body1
```

Body: one instruction per iteration — `ADD A, B`. No loads. No stores. No spills.

6.2 Counting Semantics

`range(0..n)` counts **DOWN**: DJNZ starts at B=n and decrements to 0. The element values are `n, n−1, ..., 1` — the counter itself.

This means `range(0..n).fold(0, |acc,i| acc+i)` computes **sum(1..n) = n*(n+1)/2**, not sum(0..n-1). This is intentional: counting down is the natural direction for DJNZ, and computing the triangular number is the correct mathematical result.

n	Expected result	Formula
0	0	n=0: loop doesn't run
1	1	just 1
4	10	4+3+2+1
5	15	5+4+3+2+1
10	55	10×11/2

All five verified on the Z80 binary via `TestRangeFold_E2E_SumRange`.

6.3 The Parallel Copy Bug: Caught by Dual-VM

When the range fold first compiled with correct semantics, the Z80 binary produced wrong results even though the MIR2 VM gave correct answers. This is exactly the class of **codegen bug** that would be invisible without dual-VM verification.

Root cause: The parallel copy resolver used register A as a scratch for cycle-breaking. For the A↔B cycle in the range fold trampoline, both A and B ended up as 0. The counter never loaded n.

Fix: Collect the full set of registers in the cycle (`cycleRegs`). If A is in the cycle, pick the first non-cycle 8-bit register as scratch:

```
; Correct parallel copy for A↔B cycle, scratch=C:
LD C, A     ; save n (A's original value)
LD A, B     ; A = 0 (acc init)
LD B, C     ; B = n (counter)
```

This bug was invisible to the MIR2 VM. The dual-VM assertion system caught it immediately.

Chapter 7: Compile-Time Assertions — Dual-VM Verification

7.1 The Idea

Compile-time assertions are checked as part of compilation. If an assertion fails, the build fails. No separate test runner needed.

```
fun gcd(a: u8, b: u8) -> u8 {
    while b != 0 {
        let t = b
```

```
        b = a % b
        a = t
    }
    return a
}

assert gcd(12, 8) == 4
assert gcd(100, 75) == 25
assert gcd(17, 13) == 1
```

These three assertions run every time the code compiles. If you break `gcd`, you know immediately.

7.2 Dual-VM Execution

By default, **each assertion runs twice**:

1. **MIR2 VM** — fast abstract interpreter. Runs on the SSA-form intermediate representation, before register allocation. Catches algorithm bugs.
2. **Z80 binary** — assembles the generated asm, loads into MZE (the Z80 emulator), calls the function, reads the result register. Catches codegen bugs.

```
[MIR2 optimization complete]
    ↓
assert gcd(12, 8) == 4  ← MIR2 VM: fast, ABI-agnostic
    ↓
[PBQP register allocation + Z80 codegen]
    ↓
assert gcd(12, 8) == 4  ← Z80 binary: slow, bit-exact
```

If both pass: the function is correct by construction.

7.3 What Each VM Catches

Situation	MIR2 VM	Z80 binary	What happened
`sum_range(5) = 15`	15	15	All good
`sum_range(5) = 7`	7	7	**Algorithm bug**
`sum_range(5) = 15`	15	0	**Codegen bug** (e.g. the A↔B swap)

The third row is the canonical example. The MIR2 VM is ABI-agnostic — it doesn't simulate physical registers, so it cannot observe the swap corruption. The Z80 binary check catches it immediately.

7.4 Syntax Variants

```
// Default: run on both MIR2 VM and Z80 binary
assert sum_range(5) == 15

// Only run on MIR2 VM (fast — skips Z80 assembly + emulator)
assert sum_range(5) == 15 via mir2
```

```
// Only run on Z80 binary (skips MIR2 VM)
assert sum_range(5) == 15 via z80
```

7.5 Multi-Value Assertions

```
fun divmod(a: u8, b: u8) -> (u8, u8) { ... }

assert divmod(10, 3) == (3, 1)   // quotient=3, remainder=1
```

Multi-return functions return a tuple. The assert syntax compares each return value independently.

7.6 How Z80 Asserts Work

The Z80 assert runner uses the **actual register allocation** to build the calling bootstrap:

1. Look up the compiled function's `Contract.Params` from the MIR2 module
2. Look up each param's physical location from the `AllocResult` (the output of PBQP)
3. Emit `LD <actual_reg>, arg_value` — correct even if the optimizer chose C instead of B
4. Emit `CALL funcname` + `DI / HALT`
5. Assemble with MZA, load into MZE, run
6. Read the result from the return register (determined by `Contract.Returns[0].Class`)

This means Z80 asserts are **robust to calling convention changes**: if the contract optimizer re-assigns params to different registers between compiler versions, the assert runner automatically adapts.

Chapter 8: The Optimization Pipeline

8.1 Overview of Passes

```
LUTGen             ← replace bounded pure functions with lookup tables
PropagateConstants  ← find params that are always the same value
FoldConstants       ← evaluate constant expressions (1+2 → 3)
SimplifyIdentities  ← x+0→x, x*1→x, x-x→0, etc.
ConstantCallElim    ← calls with all-const args → folded to result
DeadStoreElim       ← remove unused instructions
BranchEquiv        ← remove redundant conditional branches (VM-proved)
CondRetSink         ← convert BrIf-with-trivial-else to TermCondRet
hoistReorder        ← move Sub before Cmp for CmpSubCarry fusion
CmpSubCarry        ← replace Cmp+Sub pair with single carry-flag result
ContractOpt (PFCCO) ← interprocedural calling convention optimization
PreallocCoalesce    ← block-param ↔ block-arg register unification
PBQPAllocate        ← physical register assignment (cost-weighted)
CopyCoalesce       ← eliminate redundant moves across block boundaries
TrivialInliner      ← inline single-instruction callees (swap→0 insts)
Z80Codegen          ← emit Z80 assembly text
```

8.2 LUT Generation

Any pure function with a u8<lo..hi> ranged parameter where the range fits in ≤ 256 values is replaced with a lookup table at compile time:

```
// Before: 50-instruction sin computation
fun sin(a: u8<0..255>) -> u8 { ... }

// After LUTGen (at compile time):
// sin_lut: DB 0, 1, 3, 6, 9, 12, ...  ← 256 bytes, computed via MIR2 VM
// fun sin(a: u8) → LD HL, sin_lut / ADD HL,DE / LD A,(HL) / RET  (6 insts, ~39T)
```

8.3 BranchEquiv

The BranchEquiv pass uses the MIR2 VM to prove when a conditional branch is redundant:

Example: In the MIR2 IR for `abs_diff`, a `CmpEq` guard appears after optimizations. At the equality boundary `a == b`, both `a - b = 0` and `b - a = 0`. The branch is provably dead. Run the VM with 256 boundary inputs `(v, v)` for all v. If both sides return the same value, the branch is dead. Replace `BrIf(CmpEq)` → `Jmp`. Save 10T + 3 bytes per call.

8.4 CondRetSink and Flag Fusion: abs_diff in 4 Instructions

This five-pass optimization transforms `abs_diff` from 8 instructions to 4:

Input:

```
fun abs_diff(a: u8, b: u8) -> u8 {
    if a < b { return b - a }
    return a - b
}
```

Pass 1: CondRetSink → hoists trivial else-block, converts `BrIf` to `TermCondRet` (= RET CC) **Pass 2: SubSwapNeg** → `b - a` when we already have `a - b` → NEG **Pass 3: hoistReorder** → moves Sub before `Cmp` so carry flag contains comparison result **Pass 4: CmpSubCarry** → replaces `Cmp` with no-op (carry already set by SUB) **Pass 5: PBQP** → no interference, both map to A

Final output: 4 instructions.

```
abs_diff:   ; a=A, b=C
    SUB C       ; A = a-b, carry = (a < b unsigned)
    RET NC      ; if a >= b: return a-b (4T+10T = 14T)
    NEG         ; A = -(a-b) = b-a  (8T)
    RET         ; (10T)
```

8.5 PreallocCoalesce: DJNZ from 5 Instructions to 1

New in v4. Before PBQP allocation, PreallocCoalesce unifies block-parameter virtual registers with their corresponding block-argument virtual registers when live ranges don't overlap.

Before PreallocCoalesce (ex7_mapinplace):

```
    LD A, 1
    NEG
    ADD A, B
    LD B, A
    JRS .add2_inplace_fe_head1
```

After PreallocCoalesce:

```
    DJNZ .add2_inplace_fe_body2
```

Saving: 4 instructions, ~30T per iteration. The counter was unified with register B, allowing the back-edge to emit a single DJNZ instead of manual decrement + jump.

Impact across 6 showcase files: - `mapInPlace`: 5 instructions → 1 DJNZ - `factorial_fold`: mul16 routine eliminated entirely - `forEach/max_chain`: trampoline block removed - `fib_iter`: 3 EX DE,HL instructions eliminated - `fib_fold`: 6 redundant register moves removed

8.6 Trivial Inliner

New in v4. When a callee is a trivial function (single instruction or alias), the inliner replaces the call entirely:

```
fun swap(a: u8, b: u8) -> (u8, u8) { return (b, a) }
fun min_of(a: u8, b: u8) -> u8 { return minmax(a, b).0 }
```

- `swap(a,b).1 == a` → **zero instructions** (the compiler proves the identity statically)
- `min_of(a,b)` → `EQU minmax` (**0 bytes** — just an alias label)

8.7 Interprocedural Contract Optimization (PFCCO)

The contract optimizer searches for the best calling convention across the entire call graph:

1. **Build call graph** — topological sort (callees before callers)
2. **Candidate choices** — cartesian product of plausible register classes for each param
3. **Conflict filtering** — reject assignments where two params must share one physical reg
4. **Edge cost** — cost of crossing each call edge for a given contract pair
5. **Greedy DP** — assign contracts to minimize total T-states across all callers

Result: for a function called in a tight loop, the optimizer assigns the param directly to the register the loop already has the value in — eliminating all move instructions at the call site.

8.8 Multiply Strength Reduction

Standard power-of-2: N × `ADD HL, HL`.

Byte-boundary optimization: $* 256$ = “move low byte to high byte”:

```
; x * 256:
LD H, L     ; H = L (low byte becomes high byte)
LD L, 0     ; L = 0
; result: HL = x * 256 in 8T, 2 bytes (was: 56T, 16 bytes for 8×ADD HL,HL)
```

For * 512, * 1024 — byte-swap + remaining shifts. **Small composites** (3, 5, 6, 9): PUSH+POP+shift+add sequences (no loop).

Chapter 9: Multiple Compilation Targets

Nanz/MIR2 supports multiple output backends. The same source compiles to different targets from the same IR.

9.1 The Backend Spectrum

```
*mir2.Module
    ├── Z80Codegen          → .a80 assembly  [production]
   ├── M6502Codegen     → .s  assembly  [retro: Apple II, C64, BBC Micro]
    ├── mir2c.Codegen     → .c   file     [verification + portability]
   ├── mir2qbe.Codegen   → .ssa (QBE)    [native: x86-64, ARM64, RISC-
V]
    └── (planned) mir2llvm → LLVM IR        [future]
```

9.2 MOS 6502 Backend

New in v4. The 6502 backend compiles Nanz through the same MIR2 pipeline to 6502 assembly. **35/35 tests pass** with a dual-VM oracle (MIR2 VM vs sim6502).

```
fun abs_diff(a: u8, b: u8) -> u8 {
    if a < b { return b - a }
    return a - b
}
```

```
; 6502 output:
abs_diff:
    SEC
    SBC param_b     ; A = a - b
    BCS .done       ; if a >= b, done
    EOR #$FF        ; NEG via complement + 1
    ADC #$01
.done:
    RTS
```

Console I/O adapters for Apple II, Commodore 64, and BBC Micro are included for testing.

9.3 MIR2→C: Verification Backend

MIR2→C is a verification and portability tool. It translates MIR2 to C99 that gcc/clang can compile and run:

```c
// Generated by mir2c:
uint8_t abs_diff(uint8_t a, uint8_t b) {
    if (a < b) return b - a;
    return a - b;
}
```

Uses: Cross-checking (Z80 vs host), portability target (play-test game logic on a fast machine before deploying to Z80), reference for overflow semantics.

9.4 MIR2→QBE: Native Backend

QBE is a minimalist compiler backend that compiles `.ssa` files to x86-64, ARM64, or RISC-V native code.

```
# Generated QBE for abs_diff
export function w $abs_diff(w %a, w %b) {
@entry
    %cond =w cultw %a, %b
    jnz %cond, @then, @else
@then
    %r1 =w sub %b, %a
    ret %r1
@else
    %r2 =w sub %a, %b
    ret %r2
}
```

9.5 The `mzn` Native Compiler

New in v4. The `mzn` CLI compiles Nanz directly to native AMD64 executables:

```
# Compile via QBE (default)
mzn program.nanz

# Compile via C99
mzn -c program.nanz

# Both backends
mzn -c -q program.nanz

# Emit C99 source (inspect only)
mzn -emit-c program.nanz

# Emit QBE IL (inspect only)
mzn -emit-qbe program.nanz
```

This enables native-speed testing of Nanz programs on the development machine — the same logic runs on both Z80 and AMD64.

9.6 The Big Picture: Four Verification Targets

```
abs_diff(10, 3) == 7
    checked by:  MIR2 VM (abstract)
                 Z80 binary via MZE (physical Z80)
                 6502 binary via sim6502 (physical 6502)
```

```
C binary via gcc (host native)
QBE binary via QBE (modern native)
```

If all five agree, you can be extremely confident the function is correct.

9.7 Z80 Platform Targets

Target flag	Platform	Notes
`--target=spectrum`	ZX Spectrum 48K/128K	Default entry 0x8000, screen at 0x4000
`--target=cpm`	CP/M systems	TPA entry 0x0100, BDOS at 0x0005
`--target=agon`	Agon Light 2 (eZ80)	MOS API, VDP graphics, u24 native
`--target=generic`	Bare Z80	No platform assumptions

Chapter 10: Z80 Extern and Register Contracts

10.1 Basic @extern

```
@extern fun process(x: u8) -> void
```

The compiler assigns register classes to `process`'s parameter as normal. It will probably put x in A (ClassAcc). At the call site: `LD A, value / CALL process`.

10.2 RST Optimization

```
@extern(0x10) fun rst_16(c: u8) -> void   // RST 0x10
@extern(0x28) fun rst_40(c: u8) -> void   // RST 0x28
@extern(0xBB00) fun bdos_call(c: u8) -> void  // CALL 0xBB00
```

The compiler emits: - `RST n` for addresses that are multiples of 8 and ≤ 0x38 (1 byte, 11T) - `CALL addr` for all other addresses (3 bytes, 17T)

10.3 Annotated @extern: Precise Register Contracts

```
@extern fun LD_BYTES(@z80_a type: u8, @z80_de dest: ptr, @z80_b count: u8) -> void
// Spectrum ROM 0x0556: A=type, DE=dest, BC=length
```

The `@z80_*` annotations override PBQP for those parameters.

10.4 The ABI Is Not Fixed

Unlike traditional languages, Nanz does not have a fixed calling convention. You can observe the chosen convention in the generated assembly comment:

```
; fun abs_diff(a: u8 = A, b: u8 = C) -> u8 = A ; clobbers: F
```

Chapter 11: Verified Codegen — Showcase

23 showcase examples, all verified. Here are the highlights.

11.1 abs_diff: Five Passes to Four Instructions

```
fun abs_diff(a: u8, b: u8) -> u8 {
    if a < b { return b - a }
    return a - b
}

assert abs_diff(10, 3) == 7
assert abs_diff(3, 10) == 7
assert abs_diff(5, 5)  == 0
```

```
; fun abs_diff(a: u8 = A, b: u8 = C) -> u8 = A ; clobbers: F
abs_diff:
    SUB C       ; a - b, carry = (a < b)
    RET NC      ; a >= b: return a-b
    NEG         ; -(a-b) = b-a
    RET
```

4 instructions. Both MIR2 VM and Z80 binary verify all three asserts.

11.2 sum_range: One Instruction per Iteration

```
fun sum_range(n: u8) -> u8 {
    return range(0..n).fold(0, |acc: u8, i: u8| { return acc + i })
}

assert sum_range(0)  == 0
assert sum_range(5)  == 15
assert sum_range(10) == 55
```

Loop body: `ADD A, B` — one instruction. Verified on Z80 binary for all five test values.

11.3 mapInPlace: DJNZ Direct (PreallocCoalesce)

```
fun add2_inplace(buf: ^u8, n: u8) {
    buf.map(|x: u8| x + 2).forEach(|x: u8| { buf^ = x }, n)
}
```

```
; Loop back-edge:
    DJNZ .add2_inplace_fe_body2    ; single instruction (was 5)
```

PreallocCoalesce unified the loop counter with register B, replacing a 5-instruction decrement-branch-reload sequence with a single DJNZ.

11.4 swap: Zero Instructions

```
fun swap(a: u8, b: u8) -> (u8, u8) { return (b, a) }

assert swap(3, 7) == (7, 3)
```

```
; fun swap(a: u8 = DE, b: u8 = HL) -> (u16 = HL, u16 = DE)
swap:
    RET         ; arguments already in return positions
```

The trivial inliner proves that `swap(a,b).1 == a` at compile time — zero instructions needed.

11.5 Iterator Chain: Zero CALL Overhead

```
fun sum_filtered(buf: ^u8, n: u8) -> u8 {
    var total: u8 = 0
    buf.filter(|x: u8| x > 50).forEach(|x: u8| { total = total + x }, n)
    return total
}
```

```
sum_filtered:
    LD C, 0          ; total = 0
.loop:
    LD A, (HL)       ; load element
    INC HL
    CP 51            ; filter: x > 50 → x >= 51
    JR C, .skip      ; skip if filtered
    ADD A, C         ; total += x
    LD C, A
.skip:
    DJNZ .loop
    LD A, C          ; return total
    RET
```

No CALL for filter, no CALL for forEach body, no intermediate buffer. `total` is threaded as register C throughout.

11.6 add32: 32-bit on an 8-bit CPU

```
fun add32(a: u32, b: u32) -> u32 { return a + b }
```

```
add32:
    ADD HL, DE      ; low 16 bits
    EXX
    ADC HL, DE      ; high 16 bits + carry
    EXX
    RET
```

5 instructions using Z80 shadow register pair.

11.7 GCD: While Loop with Modulo

```
fun gcd(a: u8, b: u8) -> u8 {
    while b != 0 {
        let t = b
        b = a % b
        a = t
    }
    return a
}

assert gcd(12, 8) == 4
assert gcd(100, 75) == 25
assert gcd(17, 13) == 1
```

Compiles correctly with parallel-copy resolution at loop back-edge. Known BUG-001: extra register shuffles at loop boundaries (~25% slower than hand-written). Fix in progress via PBQP affinity edges.

11.8 Full Showcase: 23/23 PASS

#	Example	Key feature	Status
1	struct layout	Struct field offset computation	PASS
2	UFCS dispatch	`obj.method()` → direct CALL	PASS
3	zero-cost interfaces	Monomorphized dispatch	PASS
3b	interface param	Interface-typed function param	PASS
4a	abs_diff u8	4-instruction optimal	PASS
4b	abs_diff u16	16-bit variant	PASS
5	LUT popcount	Compile-time table generation	PASS
6	forEach iterator	Trampoline eliminated (v4)	PASS
7	mapInPlace	5 insts → 1 DJNZ (v4)	PASS
8	GCD	While loop with modulo	PASS
9a	factorial (recursive)	Contract: n=A	PASS
9b	factorial (fold)	mul16 eliminated (v4)	PASS
10a	fibonacci (recursive)	Recursive u16	PASS
10b	fibonacci (iterative)	Fewer clobbers (v4)	PASS
10c	fibonacci (fold)	6 moves removed (v4)	PASS
11	minmax multiret	`swap(a,b)` → RET	PASS
12	assert	Compile-time verification	PASS
13	multiret assert	Tuple return assertion	PASS
14	fold assert	For-range accumulator	PASS
15	@smc sprite	Self-modifying code	PASS
16	hello MZE	Console output	PASS
17	inline asm	Z80 asm blocks	PASS
18	console I/O	User interaction	PASS

Chapter 12: Self-Modifying Code: `@smc`

12.1 The Concept

Z80 has no immediate-mode operand for many instructions. Loading a value from memory costs ~13T. But if the value changes rarely, the compiler can **bake it into**

the instruction stream and patch the bytes when the value changes.

12.2 `@smc` Parameters

```
fun draw_sprite(@smc x: u16, @smc y: u16) {
    // x and y are baked as immediate operands:
    // LD HL, <x>  → 3 bytes, x is patched in-place
    // The compiler auto-generates set_x() and set_y() patcher functions
}
```

Generated Z80:

```
draw_sprite:
    LD HL, 0x0000       ; x baked here
draw_sprite$x$imm EQU $-2   ; patch address for x

; Auto-generated patcher:
draw_sprite_set_x:
    LD A, L
    LD (draw_sprite$x$imm), A
    LD A, H
    LD (draw_sprite$x$imm + 1), A
    RET
```

Call `draw_sprite_set_x(new_x)` to change x — the value is patched directly into the instruction bytes. Next call to `draw_sprite` uses the new value without any memory load.

12.3 Compiled Sprites

`@smc` parameters enable **compiled sprites** — each sprite frame is a hard-coded sequence of `LD (addr), val` instructions where both the address and value are baked immediates:

```
fun render_frame(@smc addr: u16) {
    // addr patched to screen position
    // Each pixel write is a single LD (HL), n instruction
}
```

For a 16×8 sprite: 346T compiled vs ~1344T LDIR. **3.8× faster.**

Chapter 13: Native Compilation: `mzn`

13.1 Overview

The `mzn` binary compiles Nanz to native AMD64 executables via two paths:

- **C99 path:** Nanz → MIR2 → C99 → gcc/clang → native binary
- **QBE path:** Nanz → MIR2 → QBE IL → qbe → native binary

13.2 Usage

```
# Compile via QBE (default, faster compile)
mzn program.nanz

# Compile via C99 (wider platform support)
mzn -c program.nanz

# Both backends (cross-check)
mzn -c -q program.nanz

# Inspect intermediate output
mzn -emit-c program.nanz      # show C99
mzn -emit-qbe program.nanz    # show QBE IL
```

13.3 VSCode Integration

Right-click any `.nanz` file for native compilation commands:

Command	What it does
Nanz: Compile to Native (C99 + QBE)	`mzn -c -q file.nanz`
Nanz: Compile to Native (C99 only)	`mzn -c file.nanz`
Nanz: Compile to Native (QBE only)	`mzn -q file.nanz`
Nanz: Emit C99 Code	Opens C99 beside source
Nanz: Emit QBE IL	Opens QBE IL beside source

13.4 Why Both Backends?

- **C99:** Maximum portability. Any platform with a C compiler. Readable output. Good for debugging MIR2 semantics.
- **QBE:** Faster compile times. Strict SSA validation (catches malformed MIR2). Native code quality closer to LLVM.

Chapter 14: Roadmap — What's Coming

Planned Features

Feature	Priority	Status
Enums	High	Parser work only — MIR2 already supports integer constants
`@error` propagation	High	CY flag pattern, depends on enums
Import system	High	Module merging exists (PL/M), parser needs `import` keyword
Type aliases	Medium	~30 LOC, pure syntactic sugar

Feature	Priority	Status
String literals	Medium	`"text"` → `^u8` null-terminated byte array
Arena allocator	Medium	Bump allocator for games/dynamic allocation
Fast multiply	Medium	Square table LUT: `f(a)*f(b) = ((a+b)²−(a−b)²)/4`
BUG-001 fix	Medium	PBQP affinity edges for block-param alignment
Compiled sprites	Low	`@smc` + attribute-only rendering
Tetris	Fun	Attribute-only, keyboard input, frame sync

Architecture Aspirations

- **Z80 signed codegen:** S^V flag for hardware i8/i16 </>=
- **WASM backend:** Nanz → MIR2 → WASM for browser demos
- **Pattern matching:** Enum destructuring with exhaustiveness checking
- **Generator syntax:** `gen { yield }` for lazy iteration

Appendix A: Grammar

```
module      = top_decl*
top_decl    = struct_decl
            | interface_decl
            | global_decl
            | fun_decl
            | '@extern' ('(' INT ')')? 'fun' fun_decl_inner
            | 'assert' assert_expr

struct_decl    = 'struct' IDENT '{' field_decl* '}'
field_decl     = IDENT ':' type ','?

interface_decl = 'interface' IDENT '{' method_name* '}'
method_name    = IDENT ','?

global_decl    = 'global' IDENT ':' type at_clause? ('=' expr)?
at_clause      = 'at' '(' expr ')'

fun_decl       = ('fun' | 'fn') fun_decl_inner
fun_decl_inner = (op_sym | IDENT ('.' IDENT)?) '(' params ')' ('->' ret_type)?
                 ('{' stmt* '}' | /* extern: no body */)
ret_type       = type | '(' type (',' type)* ')'
params         = (param (',' param)*)?
param          = reg_ann? IDENT ':' type
```

```
reg_ann        = '@z80_a' | '@z80_b' | '@z80_c' | '@z80_hl' | '@z80_de'
op_sym         = '+' | '-' | '*' | '/' | '%' | '==' | '!=' | '<' | '<='
               | '>' | '>=' | '&' | '|' | '^'

type           = '^' type
               | '[' type ';' INT ']'
               | 'u8' ('<' INT '..' INT '>')?
               | 'u16' ('<' INT '..' INT '>')?
               | 'u24' | 'u32' | 'i8' | 'i16' | 'i24' | 'i32'
               | 'f8.8' | 'f8.16' | 'f16.8' | 'f16.16' | 'f.8' | 'f.16'
               | 'bool' | 'void' | 'ptr'
               | IDENT

stmt           = var_decl | let_decl | if_stmt | while_stmt | for_stmt
               | return_stmt | 'break' | 'continue' | switch_stmt
               | asm_block | block | expr_stmt

var_decl       = 'var' IDENT ':' type at_clause? ('=' (array_init | expr))?
let_decl       = 'let' (IDENT | '(' IDENT (',' IDENT)* ')') (':' type)? '=' expr
array_init     = '[' expr (',' expr)* ']'

if_stmt        = 'if' expr block ('else' block)?
while_stmt     = 'while' expr block
for_stmt       = 'for' IDENT (':' type)? 'in'
                 (expr '[' expr? '..' expr? ']' block   // ForEachStmt (array)
                 | expr '..' expr block)                  // ForRangeStmt (int range)
return_stmt    = 'return' (expr | '(' expr (',' expr)* ')')?
switch_stmt    = 'switch' expr '{' case_clause* default_clause? '}'
case_clause    = 'case' INT ':' stmt*
default_clause = 'default' ':' stmt*
asm_block      = 'asm' ('(' IDENT ')')? '{' asm_line* '}'
block          = '{' stmt* '}'

expr_stmt      = expr ('=' expr)?

expr           = binary_expr
binary_expr    = unary_expr ((binop | 'as' type) binary_expr)*
binop          = '+' | '-' | '*' | '/' | '%' | '&' | '|' | '^'
               | '<<' | '>>' | '==' | '!=' | '<' | '<=' | '>' | '>='

unary_expr     = '-' unary_expr | '!' unary_expr | '~' unary_expr
               | '&' IDENT | postfix_expr

postfix_expr   = primary
                 ( '^'                              // dereference
                 | '[' expr ']'                    // index
                 | '.' IDENT                       // field access
                 | '.' IDENT '(' args ')'          // UFCS method call
                 | '(' args ')'                    // function call
                 | '.map' '(' lambda ')'           // iterator chain
                 | '.filter' '(' lambda ')'
                 | '.forEach' '(' lambda (',' expr)? ')'
                 | '.fold' '(' expr ',' lambda ')'
                 | '.reduce' '(' lambda ')'
                 | '.take' '(' expr ')'
                 | '.skip' '(' expr ')'
                 | '.enumerate' '(' ')'
```

```
                  | '.chain' '(' expr ')'
                  )*

primary         = INT | 'true' | 'false' | STRING
                | ('u8'|'u16'|'i8'|'i16') '(' expr ')'   // cast
                | '@ptr' '(' type ',' expr ')'
                | '@print_u8' '(' expr ')' | '@print_nl' '(' ')' | '@print_dec' '(' expr
                | '@smc' IDENT ':' type              // SMC parameter
                | 'range' '(' expr '..' expr ')'     // range source
                | '|' lambda_params '|' (block | expr)    // lambda
                | IDENT '{' field_init (',' field_init)* '}'  // struct literal
                | '(' expr (',' expr)* ')'           // parenthesized / tuple
                | IDENT

lambda_params   = (IDENT (':' type)? (',' IDENT (':' type)?)*)?
lambda          = '|' lambda_params '|' (block | expr)

assert_expr     = IDENT '(' (INT (',' INT)*)? ')' '==' (INT | '(' INT (',' INT)* ')')
                  ('via' ('mir2' | 'z80'))?

field_init      = IDENT ':' expr
args            = (expr (',' expr)*)?
```

Appendix B: Register Classes

MIR2 Register Classes

Class	Z80 register	Typical use	Cost (access)
ClassAcc	A	Accumulator, u8 return, first param	0T (ALU implicit)
ClassCounter	B	DJNZ counter, second u8 param	4T (LD A,B)
ClassPointer	HL	u16 param/return, pointer	0T (ADD HL implicit)
ClassIndex	DE	Second u16 param	4T (EX DE,HL to use in ADD)
ClassPair	BC	Third param, general pair	varies
ClassGeneral	C/D/E/H/L	Remaining 8-bit params	4T (LD A,r)
ClassDWord	HL+HL’	u32 via EXX shadow pair	~34T (ADD+EXX+ADC+EXX)
ClassFlag	F (flags)	Boolean return via carry/zero	0T at call, 4T materialize

Physical Register Availability

Register	Purpose in PBQP	Notes
A	ClassAcc	Cannot be used for indirect load to non-A
B	ClassCounter	DJNZ uses B implicitly
C	ClassGeneral	Most flexible 8-bit
D, E	ClassGeneral	DE pair for 16-bit address
H, L	ClassPointer (HL)	HL is the Z80's main 16-bit ALU reg
IX, IY	ClassIndex	Struct field access (IX+d addressing)
HL', DE', BC'	ClassShadow, ClassDWord	EXX-accessed shadow pair
A'	ClassAccShadow	EX AF,AF'
F	ClassFlag	Carry = comparison result; no LD r,F — save via PUSH AF only

Memory-Backed Registers ($F0xx)

When the interference graph has more simultaneously-live variables than physical registers, the allocator spills to absolute addresses in the $F0xx range:

```
LD ($F001), A   ; spill — 13T
LD A, ($F001)   ; reload — 13T
```

Each round-trip costs 26T. The PreallocCoalesce pass (new in v4) reduces spills by unifying block-parameter registers.

Appendix C: CLI and Tools

Compiling Nanz

```
# Compile to Z80 assembly
mz source.nanz -o output.a80

# Compile and assemble to binary
mz source.nanz -o output.bin --assemble

# Compile to TAP (ZX Spectrum tape image)
mz source.nanz --target=spectrum -o game.tap

# Emit intermediate representations
```

```
mz source.nanz --emit-hir        # HIR dump
mz source.nanz --emit-mir2       # MIR2 before optimization
mz source.nanz --emit-mir2-opt   # MIR2 after optimization
mz source.nanz --emit-asm        # Z80 assembly

# Annotate T-states in output assembly
mz source.nanz --annotate-tstates -o annotated.a80

# Compile to native AMD64
mzn source.nanz                  # via QBE (default)
mzn -c source.nanz               # via C99
```

The Toolchain

Tool	Binary	Description
MZC	`mz`	MinZ Compiler (Nanz/MinZ/PL/M-80 → Z80)
MZN	`mzn`	Native compiler (Nanz → AMD64 via C99/QBE)
MZA	`mza`	Z80 Assembler (table-driven, bracket syntax)
MZE	`mze`	Z80 Emulator (1335/1335 FUSE tests passing)
MZX	`mzx`	ZX Spectrum emulator (T-state accurate, AY sound)
MZD	`mzd`	Z80 Disassembler (IDA-like analysis, ABI propagation)
MZLSP	`mzlsp`	Language Server Protocol (diagnostics, hover, goto-def)
MZRUN	`mzrun`	Remote runner (DZRP protocol, for real hardware)
MZTAP	`mztap`	TAP file loader
MZV	`mzv`	MIR2 VM runner (breakpoints, tracing, PNG export)

Running Tests

```
cd minzc

# All packages
go test ./pkg/... -vet=off

# Specific test by name
go test ./pkg/nanz/ -run TestRangeFold_E2E_SumRange -v

# Z80 emulator tests
go test ./pkg/mir2/ -run TestMulU16ConstZ80 -v

# MOS 6502 E2E tests
go test ./pkg/mir2/ -run TestM6502 -v
```

```
# All iterator chain tests
go test ./pkg/nanz/ -run TestRange -v
```

Appendix D: What's New in v4

Features shipped since v3 (2026-03-11):

Feature	Chapter	Status
`expr as type` cast syntax	2.9	Shipped
Signed comparison (i8/i16)	3.2	Shipped
PreallocCoalesce	8.5	Shipped — 6 showcase files improved
Trivial inliner	8.6	Shipped — swap→RET, `min_of`→EQU
ForEachEdge visitor	8.1	Shipped — ~75 LOC removed
`mzn` native compiler	13	Shipped
MOS 6502 backend	9.2	Shipped — 35/35 tests
VSCode native compilation	13.3	Shipped
BUG-003 fix (ptr[i] in while)	—	Fixed (5 interacting codegen bugs)
BUG-006 fix (zero-size globals)	—	Fixed (bare label emission)
BUG-007 fix (spurious adapter LD)	—	Fixed (identity copy skip)
Multi-pass contract nudges	8.7	Shipped (mul16 rhs→DE, DJNZ→B)

Assembly improvements (v3 → v4)

Example	Before	After	Saving
ex7_mapInPlace	5-inst loop back-edge	1 DJNZ	4 insts, ~30T/iter
ex6_forEach/max_chain	trampoline + 3 insts	AND A + JRS Z	trampoline eliminated
ex9b_factorial_fold	56 lines + mul16	32 lines	mul16 routine gone
ex10b_fib_iter	3× EX DE,HL	ADD HL,HL	3 insts removed
ex10c_fib_fold	8 register shuffles	2 moves	6 moves removed
ex9a_factorial_rec	n=B contract	n=A contract	natural ABI

MinZ Compiler — modern abstractions, vintage iron, zero overhead. https://github.com/oisee/min